

Core Web Vitals (CWV)

In 2022, Angular entered a new era of evolutionary advancements known as the *Angular Renaissance*. During that period, the framework picked up momentum in web development by introducing major innovations focused on enhancing the **Developer Experience (DX)** and optimizing application performance, such as:

- A simple and modern approach to authoring Angular applications
- Improved reactivity patterns to manage application state efficiently
- The integration of **Server-Side Rendering (SSR)** techniques to improve performance

- **Angular SSR:** Renders Angular applications server-side
- **Angular service worker:** Enables Angular applications to run as **Progressive Web Applications (PWAs)** that can execute in desktop and native mobile environments

Note 1 (To Remember): SSR → Server Side Rendering

Note 2 (To Remember): SSG → Static Side Generation

- `angular.json`: The main configuration file of the Angular CLI workspace
- `package.json` and `package-lock.json`: Provide definitions of npm packages, along with their exact versions, which are needed to develop, test, and run the Angular application
- `README.md`: A README file that is automatically generated from the Angular CLI
- `tsconfig.app.json`: A TypeScript configuration that is specific to the Angular application
- `tsconfig.json`: A TypeScript configuration that is specific to the Angular CLI workspace
- `tsconfig.spec.json`: A TypeScript configuration that is specific to unit tests of the Angular application

The new version of angular, remove component from name of files:

The app folder contains the actual source code we write for our application. Developers spend most of their time inside that folder. The Angular application that was created automatically from the Angular CLI contains the following files:

- `app.component.css`: Contains CSS styles specific to the sample page. The extension of this file depends on the stylesheet format you choose when creating the application.
- `app.component.html`: Contains the HTML content of the sample page.
- `app.component.spec.ts`: Contains unit tests for the sample page.
- `app.component.ts`: Defines the *presentational logic* of the sample page.
- `app.config.ts`: Defines the configuration of the Angular application.
- `app.routes.ts`: Defines the routing configuration of the Angular application.

Components:

The files whose names start with `app.component` constitute an Angular component.

A component in Angular controls part of a web page by orchestrating the interaction of the presentational logic with the HTML content of the page, called a template.

Each Angular application has a main HTML file, named `index.html`, that exists inside the `src` folder and contains the following

HTML element:

`<body>`

`<app-root></app-root>`

`</body>`

When the Angular CLI builds an Angular application, it parses the `index.html` file and identifies HTML tags inside the `<div>` element. An Angular application is always rendered inside the `<div>` element and comprises a tree of components. When the Angular CLI finds a tag that is not a known HTML element, such as `<div>`, it starts searching through the components of the application tree.

Bootstrapping

The startup method of an Angular application is called **bootstrapping**, and it is defined in the `main.ts` file inside the `src` folder:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';
```

```
bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

The main task of the bootstrapping file is to define the component that will be loaded at application startup. It calls the *bootstrapApplication method*, passing AppComponent as a parameter to specify the starting component of the application. It also passes the appConfig object as a second parameter to specify the configuration that will be used in the application startup. The application configuration is described in the *app.config.ts file*

```
*****

import { ApplicationConfig, provideBrowserGlobalErrorListeners,
provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes)
  ]
};
```

The appConfig object *contains a providers property* to define services provided throughout the Angular application.

One of the most useful features of the component tree is that we can alter the value of a component property and inspect how the component template behaves.

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('hello-world');
  protected readonly description = "My Name is Jafar Loka";
}
```

In the *app.html*:

```
<h1>Hello, {{ title() }}</h1>
<h2>{{ description }}</h2>
```

For each component we create and we want to use it inside other component, we must declare it inside the imports-array of the other component:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { ProductList } from "../product-list/product-list";

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, ProductList],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('hello-world');
  protected readonly description = "My Name is Jafar Loka";
}
```

To define attributes for our tag, we use *attr* and *define the value in .ts-component-file*:

- For html file:

```
<p>product-list works!</p>
<div>title: {{ title() }}</div>
<p [attr.data-title]="dataTitle">Testing</p>
<p [attr.data-test]="test">For testing Only</p>
@if(products.length > 0 ) {
  <p>products: {{ products.length }}</p>
  <ul class="pill-group">
    @for (product of products; track product.id) {
      <li class="pill">{{product.title}}</li>
    }
  </ul>
} @else {
  <p>No products found!!!</p>
}
```

- For ts file:

```
import { Component, signal } from '@angular/core';
import { Product } from './product';

@Component({
  selector: 'app-product-list',
  imports: [],
  templateUrl: './product-list.html',
  styleUrls: ['./product-list.css']
})
export class ProductList {
  protected readonly title = signal('product list page')
  protected readonly dataTitle = 'test'
  products: Product[] = [
    {title: 'JLoka-01', id: 1},
    {title: 'JLoka-02', id: 2},
    {title: 'JLoka-03', id: 3},
    {title: 'JLoka-04', id: 4},
    {title: 'JLoka-05', id: 5},
  ];
}
```

Or we can use 'value-here':

```

<p>product-list works!</p>
<div>title: {{ title() }}</div>
<p [attr.data-title]="dataTitle">Testing</p>
<p [attr.data-test]="test">For testing Only</p>
*****

```

The track property, which keeps track of data changes. In our case, the track property defines the property name of the product variable that will be used to keep track of every item in the products array.

Refactor using @empty:

```

<p>product-list works!</p>
<div>title: {{ title() }}</div>
<p [attr.data-title]="dataTitle">Testing</p>
<p [attr.data-test]="test">For testing Only</p>
<p>products: {{ products.length }}</p>
<ul class="pill-group">
  @for (product of products; track product.id) {
    <li class="pill">{{product.title}}</li>
  } @empty {
    <p>No products found!!!</p>
  }
</ul>
*****

```

The other form of for-loop:

```
@for (product of products; track product.id; let variable=property) {}
```

Where property can be:

- \$count: Indicates the number of items in the array
- \$index: Indicates the index of the item in the array
- \$first/\$last: Indicates whether the current item is the first or last one in the array
- \$even/\$odd: Indicates whether the index of the item in the array is even or odd

To bind events:

```
<p>product-list works!</p>
<div>title: {{ title() }}</div>
<p [attr.data-title]="dataTitle">Testing</p>
<p [attr.data-test]="test">For testing Only</p>
<p>products: {{ products.length }}</p>
<ul class="pill-group">
  @for (product of products; track product.id) {
    <li class="pill" (click)="selectedProduct =
product">{{product.title}}</li>
  } @empty {
    <p>No products found!!!</p>
  }
</ul>

@if(selectedProduct) {
  <p>The Selected Product is: {{ selectedProduct.title }}</p>
}

-----

import { Component, signal } from '@angular/core';
import { Product } from './product';

@Component({
  selector: 'app-product-list',
  imports: [],
  templateUrl: './product-list.html',
  styleUrls: ['./product-list.css']
})
export class ProductList {
  protected readonly title = signal('product list page')
  protected readonly dataTitle = 'test'
  protected selectedProduct: Product | undefined;
  products: Product[] = [
    {title: 'JLoka-01', id: 1},
    {title: 'JLoka-02', id: 2},
    {title: 'JLoka-03', id: 3},
    {title: 'JLoka-04', id: 4},
    {title: 'JLoka-05', id: 5},
  ];
}
```

For using product-detail for product list using input-directive:

```
import { Component, input } from '@angular/core';
import { Product } from '../product-list/product';

@Component({
  selector: 'app-product-detail',
  imports: [],
  templateUrl: './product-detail.html',
  styleUrls: ['./product-detail.css']
})
export class ProductDetail {
  product = input<Product>();
}

-----

@if(product()) {
  <p style="font-weight: bold;">Product Selected: {{product()!.title}}</p>
}

-----
```

For product list:

```
import { Component, signal } from '@angular/core';
import { Product } from '../product';
import { ProductDetail } from '../product-detail/product-detail';

@Component({
  selector: 'app-product-list',
  imports: [ProductDetail],
  templateUrl: './product-list.html',
  styleUrls: ['./product-list.css']
})
export class ProductList {
  protected readonly title = signal('product list page')
  protected readonly dataTitle = 'test'
  protected selectedProduct: Product | undefined;
  products: Product[] = [
    {title: 'JLoka-01', id: 1},
    {title: 'JLoka-02', id: 2},
    {title: 'JLoka-03', id: 3},
    {title: 'JLoka-04', id: 4},
    {title: 'JLoka-05', id: 5},
  ];
}

-----
```

```
<p>product-list works!</p>
<div>title: {{ title() }}</div>
<p [attr.data-title]="dataTitle">Testing</p>
<p [attr.data-test]="test">For testing Only</p>
<p>products: {{ products.length }}</p>
<ul class="pill-group">
  @for (product of products; track product.id) {
    <li class="pill" (click)="selectedProduct =
product">{{product.title}}</li>
  } @empty {
    <p>No products found!!!</p>
  }
</ul>

<app-product-detail [product]="selectedProduct"></app-product-detail>
```
